

Assignment 10

Q1 Show that EQ_{CFG} is co-Turing-recognizable

A1

We just need to show $\overline{EQ_{CFG}}$ is Turing-recognizable.

TM M recognizes $\overline{EQ_{CFG}}$

$M =$ "On input w

1. Check if the form of w is $\langle G_1, G_2 \rangle$ where G_1, G_2 are two valid CFGs. Accept if not.
2. Convert G_1, G_2 into Chomsky Normal Form.
3. For $i = 1, 2, 3, \dots$, repeat (4)(5).
4. Generate a list of all length i strings, then run D_{A-CFG} on $\langle G_1, s \rangle$ and $\langle G_2, s \rangle$ for each string s in the list. (As we know, A_{CFG} is decidable, obviously there exists a decider that can decide it. We assume the decider is D_{A-CFG} .)
5. If for some s , D_{A-CFG} gives different results for $\langle G_1, s \rangle$ and $\langle G_2, s \rangle$, accept."

Q2 If $A \leq_m B$ and B is a regular language, does that imply that A is a regular language? Why or why not?

A2

The answer is No.

Consider the following languages A and B defined over the alphabet $\Sigma = \{a, b\}$:

- $A = \{a^n b^n \mid n \geq 0\}$
- $B = \{b\}$

Define the function $f : \Sigma^* \rightarrow \Sigma^*$ as:

$$f(w) = \begin{cases} b & \text{if } w \in A, \\ a & \text{if } w \notin A. \end{cases}$$

- **Observation:** $w \in A$ if and only if $f(w) = b$, which holds true if and only if $f(w) \in B$.
- Since B is regular language (being finite), if A were also regular, f would be a computable function.
- However, the language A is context-free and not regular, evidenced by the fact that it requires matching numbers of a 's and b 's which cannot be recognized by a finite automaton.

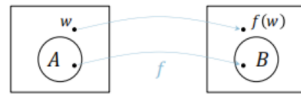
Thus, the fact that B is regular does not imply A is regular, despite A being mappable to B through f . Hence, A is context-free and Turing-decidable but not regular.

Q3 Show that if A is Turing-recognizable and $A \leq_m \bar{A}$, then A is decidable.

A3

We can show that $\bar{A} \leq_m A$:

Since $A \leq_m \bar{A}$, there exists a function $f : \Sigma^* \rightarrow \Sigma^*$ where $w \in A$ iff $f(w) \in \bar{A}$.



Therefore, for any $m \notin A$, $f(m) \notin \bar{A}$; that is to say, $m \in \bar{A}$ iff $f(m) \in A$.

Thus f can be used for the mapping reduction $\bar{A} \leq_m A$.

Then,

A is T-recognizable and $\bar{A} \leq_m A$ implies that $\bar{A}$ is T-recognizable.

Hence,

$\bar{A}$ is T-recognizable and A is T-recognizable implies that A is decidable.

Q4 Let $T = \{\langle M \rangle \mid M \text{ is a TM that accepts } w^R \text{ whenever it accepts } w\}$. Show that T is undecidable.

A4

Suppose, for contradiction, that T is decidable. Let D be a decider for T .

We construct a TM S that decides

$$A_{TM} = \{\langle K, s \rangle \mid K \text{ accepts } s\}.$$

$S =$ "On input $\langle K, s \rangle$:

1. Check whether K is a Turing machine and s is a string. Reject if not.
2. Construct a new TM N as follows.

$N =$ "On input x :

1. If $x = 01$, accept.
2. If $x \neq 01$, simulate K on s .

If K accepts s , accept.

If K rejects s , reject.

If K loops on s , loop."

3. Run D on $\langle N \rangle$.

4. If D accepts, accept. If D rejects, reject."

Now we prove correctness.

If K accepts s , then for every input $x \neq 01$, N accepts x . Also, N accepts 01 . Hence

$$L(N) = \Sigma^*.$$

Therefore, whenever N accepts a string w , it also accepts w^R . So

$$\langle N \rangle \in T.$$

Thus D accepts $\langle N \rangle$, and S accepts $\langle K, s \rangle$.

If K does not accept s , meaning K either rejects or loops on s , then N only accepts 01 . Hence

$$L(N) = \{01\}.$$

But

$$01 \in L(N)$$

and

$$(01)^R = 10 \notin L(N).$$

Therefore, N does not satisfy the property that it accepts w^R whenever it accepts w . So

$$\langle N \rangle \notin T.$$

Thus D rejects $\langle N \rangle$, and S rejects $\langle K, s \rangle$.

Therefore,

$$\langle K, s \rangle \in A_{TM} \iff \langle N \rangle \in T.$$

So if D decides T , then S decides A_{TM} . This is impossible because A_{TM} is undecidable.

Hence, T is undecidable.

Q5 Show that A is Turing-recognizable iff $A \leq_m A_{TM}$.

A5

- If $A \leq_m A_{TM}$, A is Turing-recognizable since A_{TM} is Turing-recognizable.
- If A is Turing-recognizable, we suppose TM T recognizes A .

We can construct a function $f(w) = \langle T, w \rangle$, and $f(w) \in A_{TM}$ iff $w \in A$.

So that we have $A \leq_m A_{TM}$.

To sum up, A is Turing-recognizable iff $A \leq_m A_{TM}$.

Q6 Show that the language $T = \{ \langle M \rangle \mid M \text{ halts on empty input} \}$ is undecidable.

A6

Prove by contradiction.

Assume T is decidable via a decider D .

Show that $HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$ can be reducible to T :

Construct TM S deciding $HALT_{TM}$:

$S =$ "On input $\langle M, w \rangle$

1. Define a new TM $M' =$

"On input ϵ

1. Simulate M on w .

2. If M halts on w , M' halts.

3. If M loops on w , M' loops."

2. If M halts on w : M' halts on ϵ , so $\langle M' \rangle \in T$.

If M does not halt on w : M' loops on ϵ , so $\langle M' \rangle \notin T$.

3. Use D to check whether $M' \in T$:

If D accepts, then M halts on w .

If D rejects, then M does not halt on w ."

If D was a decider for T , then S would be a decider for $HALT_{TM}$. Contradiction. So T must be undecidable.

Q7

Use Rice's Theorem to prove undecidability of each of the following languages:

a. $INFINITE_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is an infinite language} \}$.

b. $\{ \langle M \rangle \mid M \text{ is a TM and } 1011 \in L(M) \}$.

c. $ALL_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) = \Sigma^* \}$.

A7

Recall:

Rice's Theorem

The fact that $REGULAR_{TM}$ is undecidable is actually a special case of a more general theorem

A **property** of recognizable languages is any set of recognizable languages (a predicate that a language satisfies)

Def. A property P of a language is **trivial** if:

1. All languages have the property
2. No languages have the property

All other properties are nontrivial.

Def. A property P is **nontrivial** if some languages have the property and others don't. It contains:

1. Some, but not all TM descriptions
2. Whenever $L(M_1) = L(M_2)$, then $\langle M_1 \rangle \in P$ iff $\langle M_2 \rangle \in P$

Rice's Theorem: Every nontrivial property P is undecidable

This implies that the problems of determining whether the language of a TM is

- context free,
- a decidable language,
- a finite language

Are all undecidable!

Because determining **any property** of the languages recognized by Turing machines is undecidable

a)

$INFINITE_{TM}$ contains some, but not all TM descriptions (it contains TM "Accept any string" but not contains TM "Accept ϵ only").

If two TMs M_1, M_2 have $L(M_1) = L(M_2)$, then $L(M_1)$ is infinite iff $L(M_2)$ is infinite; then

$$\langle M_1 \rangle \in INFINITE_{TM} \text{ iff } \langle M_2 \rangle \in INFINITE_{TM}.$$

Rice's theorem implies that $INFINITE_{TM}$ is undecidable.

b)

Let B denote the language.

B contains some, but not all TM descriptions (it contains TM "Accept any string" but not contains TM "Reject any string").

If two TMs M_1, M_2 have $L(M_1) = L(M_2)$, then $1011 \in L(M_1)$ iff $1011 \in L(M_2)$; then

$$\langle M_1 \rangle \in B \text{ iff } \langle M_2 \rangle \in B.$$

Rice's theorem implies that B is undecidable.

c)

ALL_{TM} contains some, but not all TM descriptions (it contains TM "Accept any string of alphabet Σ " but not contains TM "Reject any string").

If two TMs M_1, M_2 have $L(M_1) = L(M_2)$, then $L(M_1) = \Sigma^*$ iff $L(M_2) = \Sigma^*$; then

$$\langle M_1 \rangle \in ALL_{TM} \text{ iff } \langle M_2 \rangle \in ALL_{TM}.$$

Rice's theorem implies that ALL_{TM} is undecidable.

Q8

True or False

- a. $2n = O(n)$
- b. $n^2 = O(n \log^2 n)$
- c. $3^n = 2^{O(n)}$
- d. $n = o(2n)$
- e. $n^{1/10} = o(\log n)$
- f. $\frac{n^2}{1000} - 10000n = \Omega(n^2)$

A8

a. True b. False c. True d. False e. False f. True

Q9

Fill out the table described in the polynomial time algorithm for context free language recognition for string $w = baba$ and CFG G :

$$\begin{aligned} S &\rightarrow RT \\ R &\rightarrow TR \mid a \\ T &\rightarrow TR \mid b \end{aligned}$$

A9

Algorithm:

Example 3: CFL.(2)

Theorem: CFL $\in$ P

Idea: Use **Dynamic Programming** to:

1. solve smaller substrings and save the solutions to the smaller subproblems!
2. Use the solutions to the smaller subproblems to generate larger substrings!
 - Let $w = w_1 w_2 \dots w_n$ be the string to be generated
 - Create a table of size $n \times n$:
 - for $i \leq j$, the (i, j) th entry contains the variables that generate $w_i w_{i+1} \dots w_j$
 - First fill the entries for strings of length 1, then length 2 and so on...
 - Eg., If the algorithm has determined which variables generate all substrings of length k
 - For length $k + 1$ the algorithm splits each substring in all possible ways and checks the table
 - For $A \rightarrow BC$ check whether B generates the first bit and C generates the second
 - For length 1 the algorithm checks all the rules $A \rightarrow b$

The final table is as follows:

	1	2	3	4
1	T	R, T	S	S, R, T
2		R	S	S
3			T	R, T
4				R

The starting variable $S \in \text{table}(1, 4)$, thus CFG G generates the string $w = baba$.

Q10 Show that P is closed under union, concatenation and complementation.

A10

a)

Assume $A \in P$, $B \in P$, and T_A decides A , T_B decides B .

Denote $t_M(n)$ as the upper bound of the runtime of TM M . It's easy to see $t_{T_A}(n)$ and $t_{T_B}(n)$ are both polynomial.

For union operation, we build a TM $T_\cup$ to decide $A \cup B$.

$T_\cup =$ "On input s :

1. Run T_A on s . Accept if T_A accepts.
 2. Run T_B on s . Accept if T_B accepts.
- Reject if T_A and T_B both reject."

We can calculate that

$$t_{T_\cup}(n) \leq t_{T_A}(n) + t_{T_B}(n)$$

Thus, $A \cup B \in P$.

b)

For concatenation operation, we build a TM $T_\circ$ to decide $A \circ B$.

$T_\circ =$ "On input s :

1. Assume s is $s_1 s_2 \dots s_n$, where n is the length of s .
Repeat the following steps for iteration number i , starts from 1, end at n .
Run T_A on $s_1 s_2 \dots s_i$, run T_B on $s_{i+1} s_{i+2} \dots s_n$.
Accept if both T_A and T_B accept.
2. Reject if T_A and T_B haven't both accepted once.

We can calculate that

$$t_{T_{\circ}}(n) \leq n \times (t_{T_A}(n) + t_{T_B}(n))$$

Thus, $A \circ B \in P$.

c)

For complementation operation, we build a TM $T_{\neg}$ to decide $\overline{A}$.

$T_{\neg}$ = "On input s :

1. Run T_A on s .

Accept if T_A rejects, reject if T_A accepts.

We can calculate that

$$t_{T_{\neg}}(n) \leq t_{T_A}(n)$$

Thus, $\overline{A} \in P$.

To sum up, P is closed under union, concatenation and complementation.

Q11

Let $CONNECTED = \{ \langle G \rangle \mid G \text{ is a connected undirected graph} \}$. Show that this language is in P.

HINT: In lecture 8 a TM to decide the language was provided.

A11

Construct the TM M decides

$$CONNECTED = \{ \langle G \rangle \mid G \text{ is a connected undirected graph} \}.$$

M = "On input $\langle G \rangle$, the encoding of a graph $\langle G \rangle$:

1. Select the first node of G and mark it.
2. Repeat the following stage until no new nodes are marked:
 3. For each node in G , mark it if it is attached by an edge to a node that is already marked.
4. Scan all the nodes of G to determine whether they all are marked. If they are, accept; otherwise, reject."

step 1: $O(1)$

step 2: $O(n)$ iterations

step 3: $O(n^2)$ steps

step 4: $O(n)$ steps

The runtime of TM M is $O(n^3)$.

Hence $CONNECTED$ is in P .

Q12 Show that $ALL_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^* \}$ is in P.

A12

Give TM D that decides ALL_{DFA} .

$D =$ "On input $\langle A \rangle$

1. Mark start state. $O(1)$
2. Repeat until no new state to mark: $O(n)$
 Mark every state that has an incoming arrow from a previously marked state. $O(n^2)$
3. Scan all marked states to check if they're all accept states. Accept if they are. Reject if some are not." $O(n)$

The algorithm takes $O(1) + O(n) \times O(n^2) + O(n) = O(n^3)$ to decide ALL_{DFA} , which means ALL_{DFA} is in P.

Q13 Show that $EQ_{DFA} \in P$

A13

Give TM D that decides EQ_{DFA} .

$D =$ "On input $\langle A \rangle$

1. Mark start state. $O(1)$
2. Repeat until no new state to mark: $O(n)$
 Mark every state that has an incoming arrow from a previously marked state. $O(n^2)$
3. Scan all marked states to check if no accept state is marked. Accept if it is. Reject if not." $O(n)$

D takes $O(n^3)$ to run.

Give TM T decides EQ_{DFA} .

$T =$ "On input $\langle A, B \rangle$

1. Construct DFA C where

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B)).$$

This DFA can be constructed within polynomial time.

2. Run D on $\langle C \rangle$. $O(n^3)$

3. Accept if D accepts. Reject if D rejects." $O(n)$

Hence the overall runtime is polynomial. Thus $EQ_{\text{DFA}} \in P$.